Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Sector No. 26, Pradhikaran, Nigdi, Pune 411044

An Autonomous Institute Approved By AICTE and Affiliated To SPPU, Pune

Department Of Information Technology

Software Engineering & Modelling Laboratory

[Course: BIT25PC05]

Mini Project Report

of

T. Y. B. Tech

Academic Year: 2026-27

Semester - I

On

VoltGrid: EV Charging Network & Fleet Energy Management System

By

PRN No.	Name of the Student
124B1F054	Aryan Suryawanshi
124B1F056	Soham Kulkarni
124B1F144	Prathmesh Naik
124B1F050	Sarang Kolekar

CERTIFICATE

This is to certify that the project report entitled VoltGrid: EV Charging Network & Fleet Energy Management System, is successfully completed by Aryan Suryawanshi, Soham Kulkarni, Prathmesh Naik, and Sarang Kolekar under the Mini Project course of Software Engineering and Modelling Laboratory in the Third Year of the Information Technology programme. The work fulfils the course requirements for Semester V, Academic Year 2026–27.

Internal Project Guide

INDEX

Chapter No.	Title of the Content	Page No.
1	**Introduction**	1
1.1	Problem Statement	1
1.2	Objectives	1
1.3	Scope of the Project	1
2	**Software Requirement Specification (SRS)**	2–3
2.1	Purpose of Project	2
2.2	Functional Requirements	2
2.3	Non-Functional Requirements	3
2.4	User Requirements	3
3	**System Analysis & OOMD**	4
3.1	System Overview	4
3.2	Identification of Actors and Objects	4
3.3	Class Identification	4
4	**UML Structural & Behavioural Modelling**	5–9
4.1	Use Case Diagram	5
4.2	Class Diagram	6
4.3	Sequence Diagram	7
4.4	State Chart Diagram	8
4.5	Activity Diagram	9
5	**Software Testing**	10–11
5.1	Test Cases	10
5.2	Test Results	11
6	**Conclusion & Future Scope**	12
6.1	Conclusion	12
6.2	Future Enhancements	12
7	**Project Functionality Screenshots**	13-15

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

EV charging services often separate station discovery, slot booking, charging operations, fleet planning, and fault handling. Drivers cannot easily compare available connectors or reserve a reliable time slot, while operators need a single view of bookings and charger condition. VoltGrid brings these workflows into one role-based web application. The requirements were developed in the team's SRS [1].

1.2 Objectives

- Allow drivers to discover nearby stations, filter connectors, and request charging slots.
- Let operators manage stations, chargers, connector availability, and booking approvals.
- Provide fleet managers with vehicle, driver, schedule, billing, and analytics views.
- Give administrators visibility into users, stations, reports, and platform activity.
- Protect data by role through authenticated routes and database row-level security.

1.3 Scope of the Project

The implemented prototype uses a React and Vite client with Supabase Authentication and PostgreSQL. It includes four portals, persisted records, a booking approval workflow, wallet and fault interfaces, and a software kiosk simulator. The kiosk represents charger behaviour; it is not a connection to physical charging hardware. Payment gateway integration and production-grade charger control remain outside the present implementation.

CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

2.1 Purpose of Project

VoltGrid provides a shared system for EV drivers, station operators, fleet managers, and platform administrators. Its requirements aim to make charger discovery and reservation traceable while keeping each role's data and actions appropriately limited.

2.2 Functional Requirements

Table 2.1 Functional requirements

ID	Requirement
FR-01	Register, authenticate, recover accounts, and direct users to role-specific portals.
FR-02	Display stations and connectors with location, availability, and search or filter controls.
FR-03	Create a pending reservation, hold the slot, and let an operator approve or cancel it.
FR-04	Display charging sessions, usage, transaction, and wallet information.
FR-05	Allow operators to manage station assets, reservations, faults, and revenue views.
FR-06	Allow fleet managers to manage vehicles, drivers, schedules, analytics, and billing.
FR-07	Allow administrators to review users, stations, reports, analytics, and settings.
FR-08	Support fault tickets, notifications, reviews, and community content.

The booking workflow is central: a driver submits a station, connector, and slot; the system saves a pending reservation; an operator decision changes its status and becomes visible to the driver. Database constraints and policies protect these records beyond the user interface.

2.3 Non-Functional Requirements

Table 2.2 Non-functional requirements

Quality	Requirement / design response
Security	Supabase Authentication, role guards, and PostgreSQL row-level security restrict access.
Usability	Responsive screens and clear loading, empty, and error states support desktop and mobile use.
Reliability	Database transactions and reservation rules keep booking state consistent.
Maintainability	Pages are separated by role; domain-specific API modules isolate database operations.
Performance	Route-level lazy loading limits the initial client bundle; station distance is calculated locally.
Accessibility	Radix UI primitives support keyboard and assistive technology interaction.

2.4 User Requirements

Table 2.3 User requirements by role

Actor	Primary needs
Driver	Find chargers, reserve a slot, track a session, view wallet activity, and report faults.
Operator	Maintain stations and connector status, decide reservations, and handle faults.
Fleet manager	Plan charging for multiple vehicles and review usage and billing.
Administrator	Monitor users, assets, content, and network-level reports.

External dependencies include a browser with geolocation permission for nearby station sorting and a reachable Supabase project. The current telemetry interface is simulated. Any real payment or charger integration requires further implementation and validation.

CHAPTER 3: SYSTEM ANALYSIS & OOMD

3.1 System Overview

The client is a React single-page application. Authentication identifies the user and resolves a role. Each portal calls domain API modules, which query Supabase PostgreSQL. Row-level security enforces who may read and write records. A reservation is created with PENDING status, and an operator can move it to RESERVED or CANCELLED. Notifications communicate relevant changes.

3.2 Identification of Actors and Objects

Table 3.1 Actors and core domain objects

Actors	Core objects
Driver	Profile, Station, Charger, Connector, Reservation, Session, Wallet, Ticket
Station operator	Station, Charger, Connector, Reservation, Ticket
Fleet manager	Fleet, Vehicle, Driver, Schedule, Invoice
Administrator	Profile, Station, Report, Marketplace listing

3.3 Class Identification

The conceptual domain classes are UserProfile, Station, Charger, Connector, Reservation, ChargingSession, Transaction, FaultTicket, FleetVehicle, and Notification. A station contains chargers; a charger exposes connectors; a reservation links a driver to a connector and time slot; a session records the charging activity. The class diagram on page 9 presents the key relationships. In implementation, these concepts are represented by database tables and JavaScript API modules rather than a single object-oriented server model.

CHAPTER 4: UML STRUCTURAL & BEHAVIOURAL MODELLING

4.1 Use Case Diagram

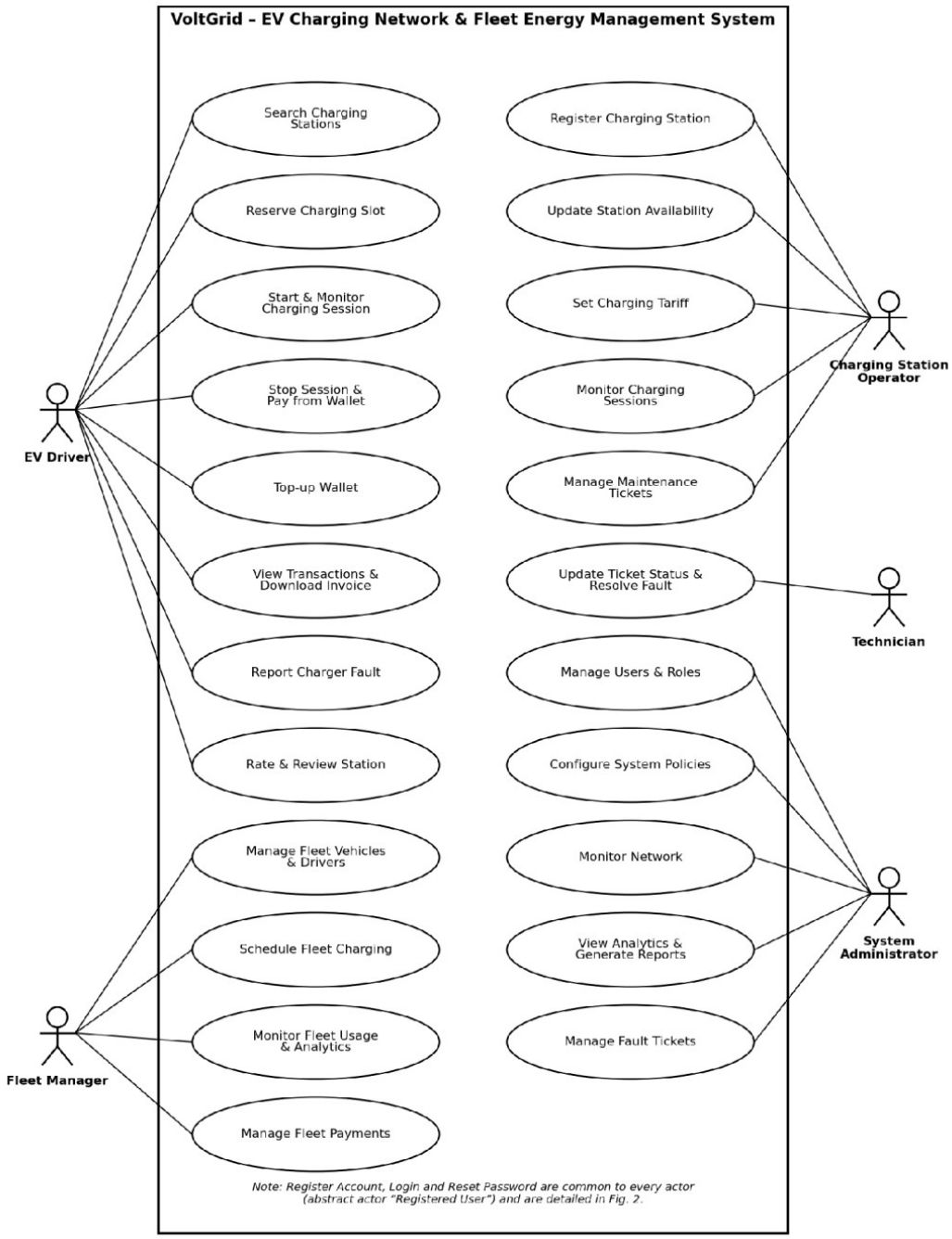


Figure 4.1 Submitted overview use case model [2]

The submitted use case model [2] covers the intended platform, including a technician role. The implemented prototype has driver, operator, fleet manager, and administrator portals; a dedicated technician portal remains planned.

4.2 Class Diagram

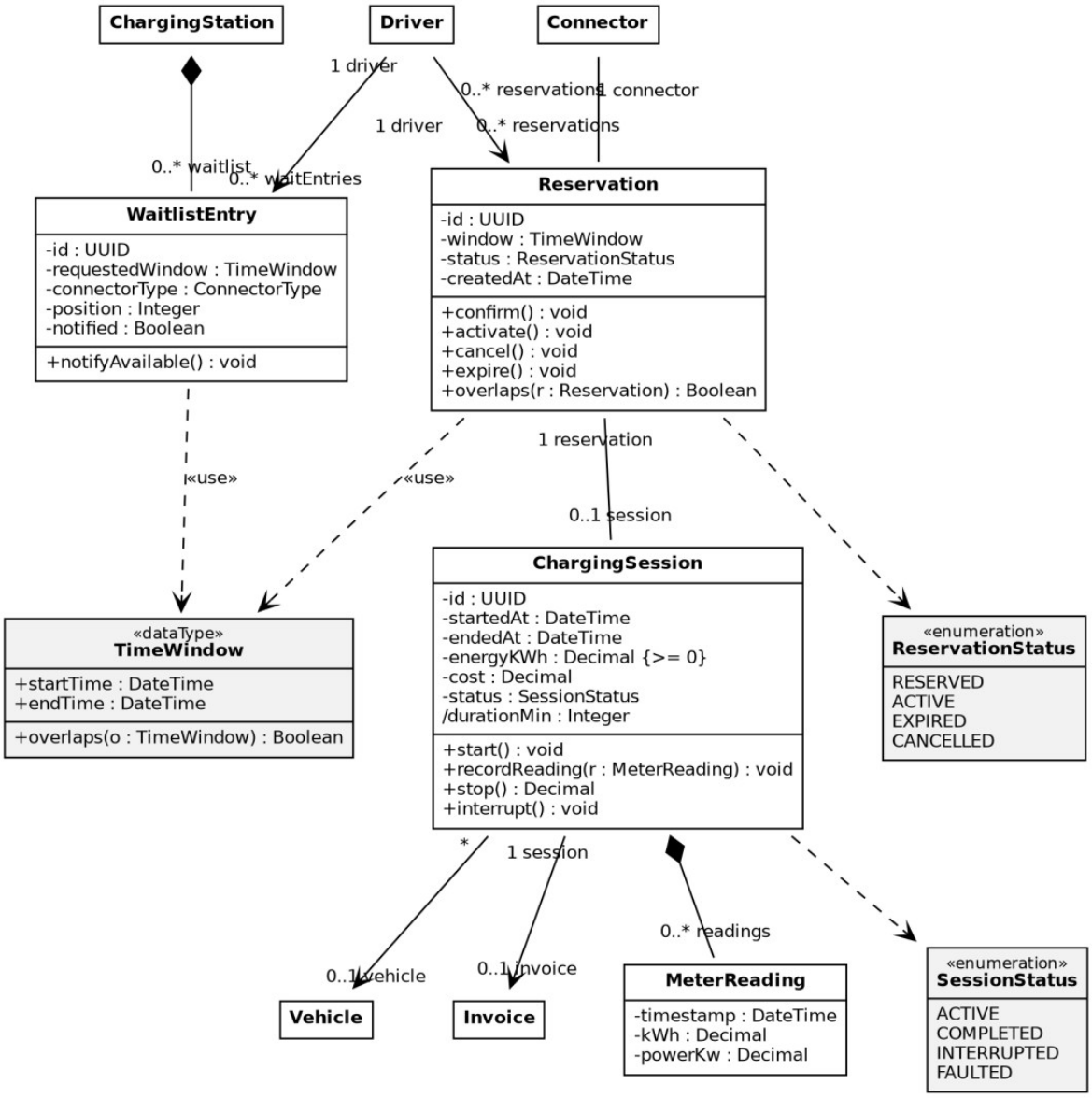


Figure 4.2 Submitted reservation and session design model [3]

The submitted design class model [3] represents reservation, waitlist, connector, and session relationships. It predates the implemented PENDING approval state and includes planned domain operations; the current implementation uses Supabase tables and client API modules.

4.3 Sequence Diagram

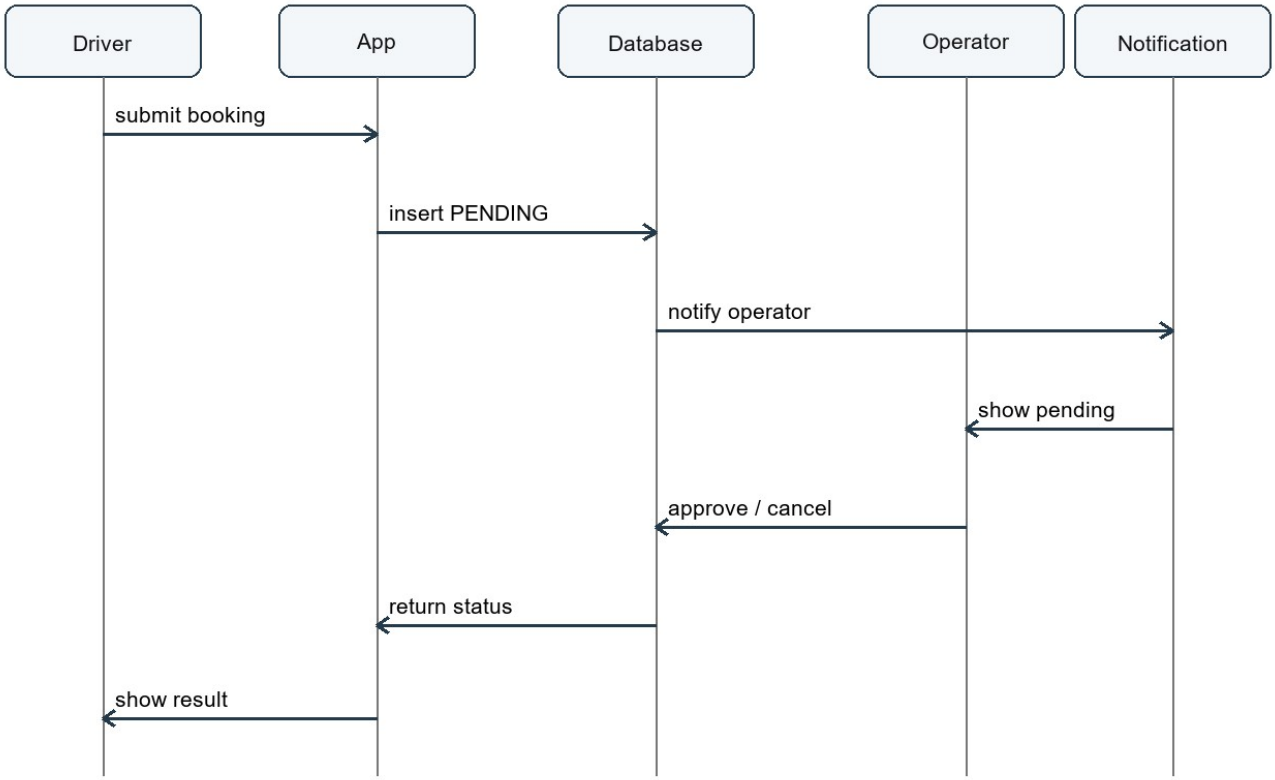


Figure 4.3 Reservation approval sequence

The database persists the pending request and broadcasts an operator notification. The operator decision updates the status shown to the driver.

4.4 State Chart Diagram

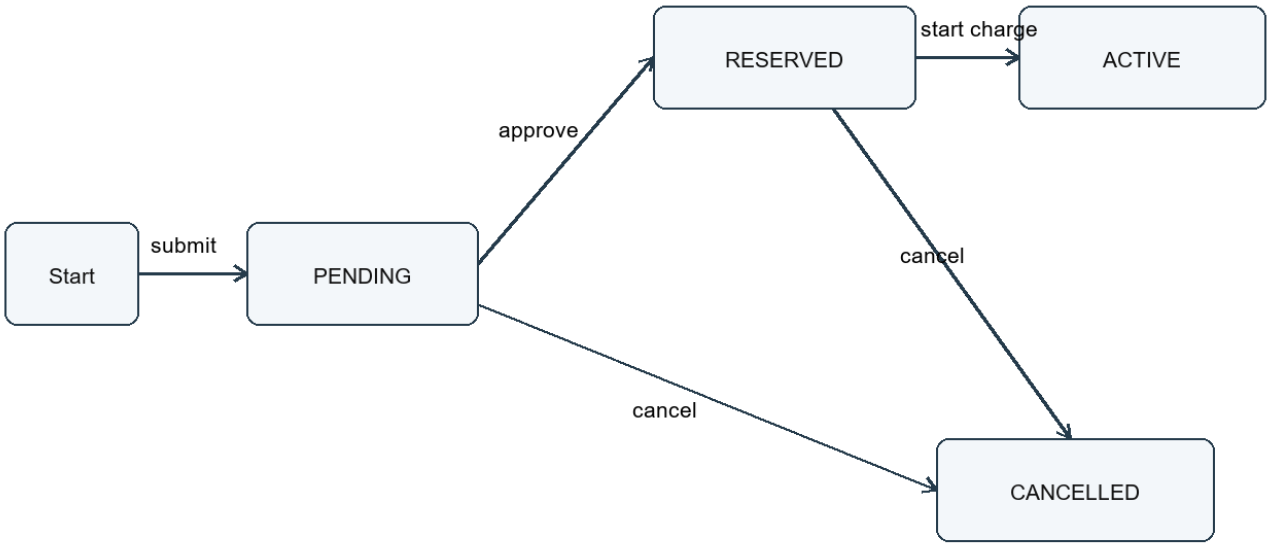


Figure 4.4 Reservation state transitions

A new request is pending until an operator approves or cancels it. An approved reservation may become active when charging starts. The diagram shows the main implemented booking transitions.

4.5 Activity Diagram

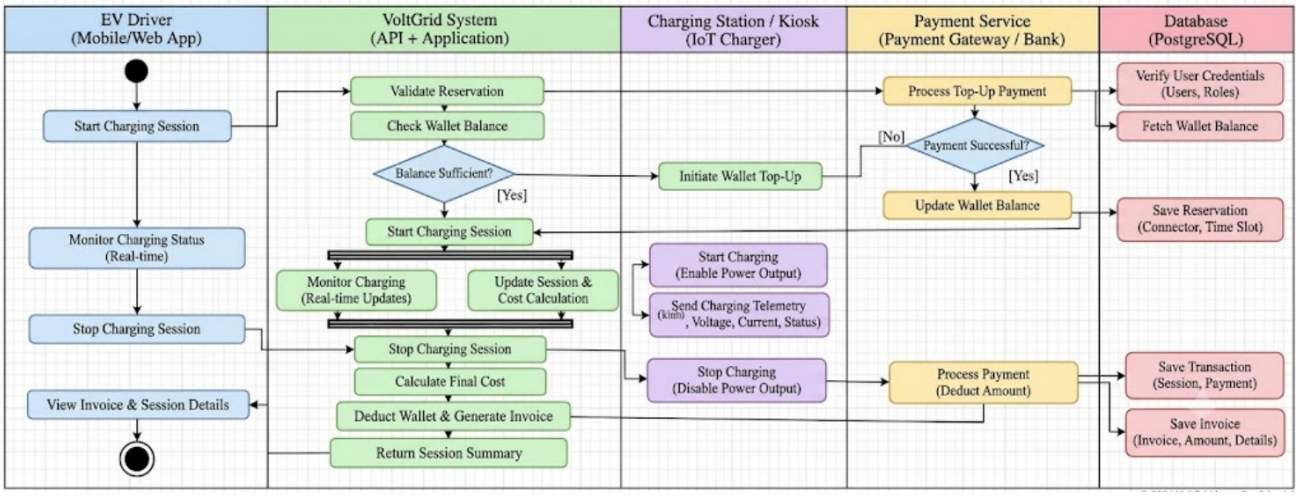


Figure 4.5 Submitted charging activity model [4]

The submitted activity model [4] includes payment gateway and physical charger steps as design intent. In the prototype, booking approval is implemented, payment is simulated, and charging hardware is represented by a kiosk emulator.

CHAPTER 5: SOFTWARE TESTING

5.1 Test Cases

The table defines checks tied to the implemented flows. Automated results are summarized on the next page; browser and external-service flows require separate manual verification.

Table 5.1 Test cases

ID	Scenario	Expected result
TC-01	Valid sign-up with role	Profile created with selected role
TC-02	Cross-role URL access	Unauthorized portal blocked
TC-03	Station search and distance	Stations sorted or filtered correctly
TC-04	Create booking	PENDING reservation and held slot
TC-05	Operator approves booking	Status becomes RESERVED
TC-06	Operator cancels booking	Status becomes CANCELLED
TC-07	Driver reads another user booking	Database denies access
TC-08	Fault report submission	Ticket recorded and visible to operator
TC-09	Wallet transaction	Balance and ledger update consistently
TC-10	Fleet schedule write	Schedule saved for authorized manager
TC-11	Password reset email flow	Recovery link and new password work
TC-12	OAuth role onboarding	Role selection completes after provider login

5.2 Test Results

Project records in SUPABASE_HANDOVER.md report 82 successful live integration checks on 21 August 2026 across role isolation, writes, views, and admin functions. That evidence applies to the tested database state and does not establish production readiness. The repository also includes automated QA scripts and build/lint commands.

Table 5.2 Recorded test evidence

Area	Recorded evidence	Status
Authentication and role isolation	Live sign-up, profile, and role policy checks	Verified in project record
Booking and portal writes	Live checks exercised reservation and domain writes	Verified in project record
Derived database views	Checked recomputation after changes	Verified in project record
OAuth and password reset	Handover states these need browser or inbox verification	Pending manual test
Physical charger and payment gateway	No physical charger or live payment integration	Out of prototype scope

The kiosk interface is a software emulator. Session state and some analytics demonstrations are simulated or snapshot based, as documented in the handover. The report therefore does not claim real charger telemetry, live payment processing, or fully validated OAuth recovery.

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Conclusion

VoltGrid demonstrates a role-based EV charging platform that joins station discovery, reservation approval, operator management, fleet planning, fault reporting, and administration in one application. The implemented React client and Supabase database provide persistent records and server-enforced row-level access. The booking workflow shows how a driver request can be reviewed by an operator while keeping status visible across portals.

The prototype addresses the course objectives of requirements analysis, object identification, UML modelling, and testing. It remains a software prototype: charging hardware is emulated, and some end-to-end integrations require further validation.

6.2 Future Enhancements

- Integrate physical chargers through a standard charger communication protocol and verify live telemetry.
- Connect a real payment gateway with auditable settlement and failure handling.
- Complete browser-based OAuth and password-reset tests and strengthen administrator provisioning.
- Expand automated end-to-end tests for booking conflicts, cancellations, faults, and billing.
- Replace snapshot analytics and simulated state with event-derived production metrics.

REFERENCES

[1] VoltGrid project team, “Software Requirements Specification,” SEML Assignment 2, PCCOE Department of Information Technology.

[2] VoltGrid project team, “VoltGrid Use Cases,” SEML Assignment 8, PCCOE Department of Information Technology.

[3] VoltGrid project team, “VoltGrid Design Model,” SEML Assignment 9, PCCOE Department of Information Technology.

[4] VoltGrid project team, “Activity Diagrams,” SEML Assignment 7, PCCOE Department of Information Technology.

CHAPTER 7: PROJECT FUNCTIONALITY SCREENSHOTS

7.1 Driver: Station Discovery and Booking

Representative interface captures from project test data.

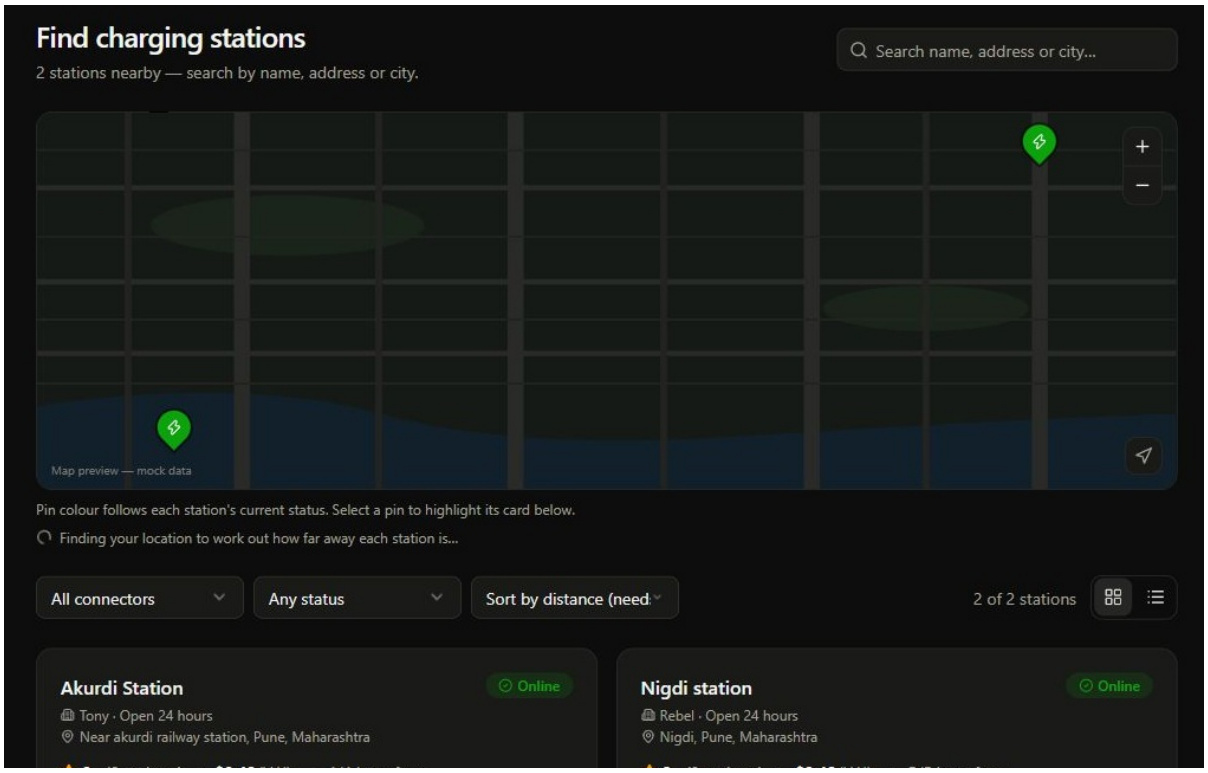


Figure 7.1 Driver station search, map, and availability

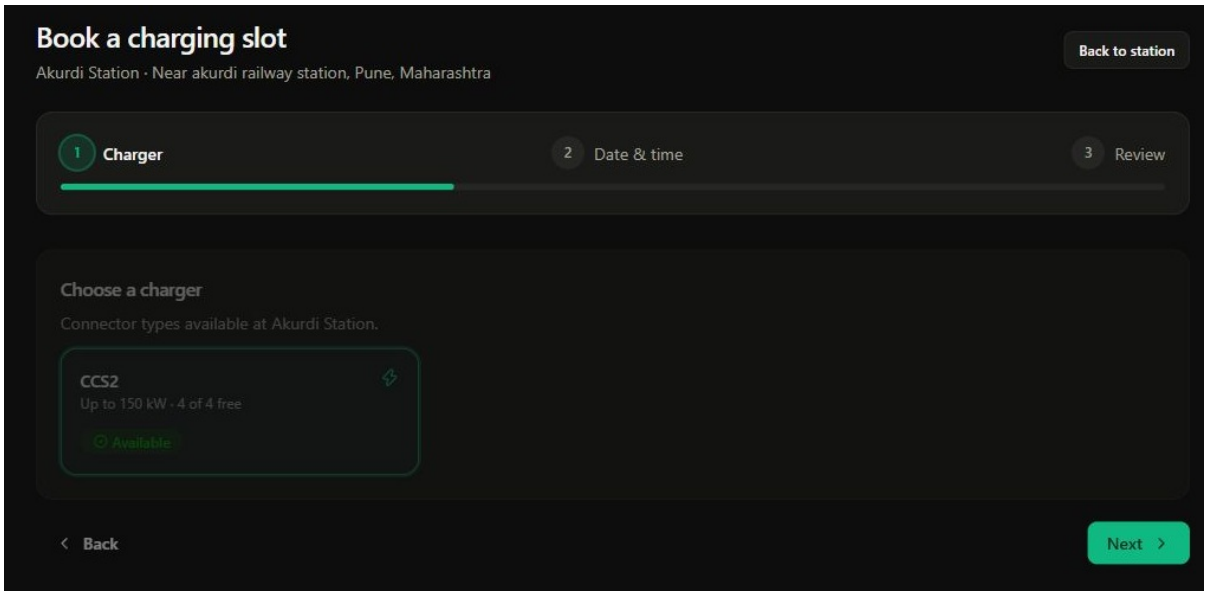


Figure 7.2 Charging slot booking flow

CHAPTER 7: PROJECT FUNCTIONALITY SCREENSHOTS

7.2 Operator: Station and Connector Management

Representative interface captures from project test data.

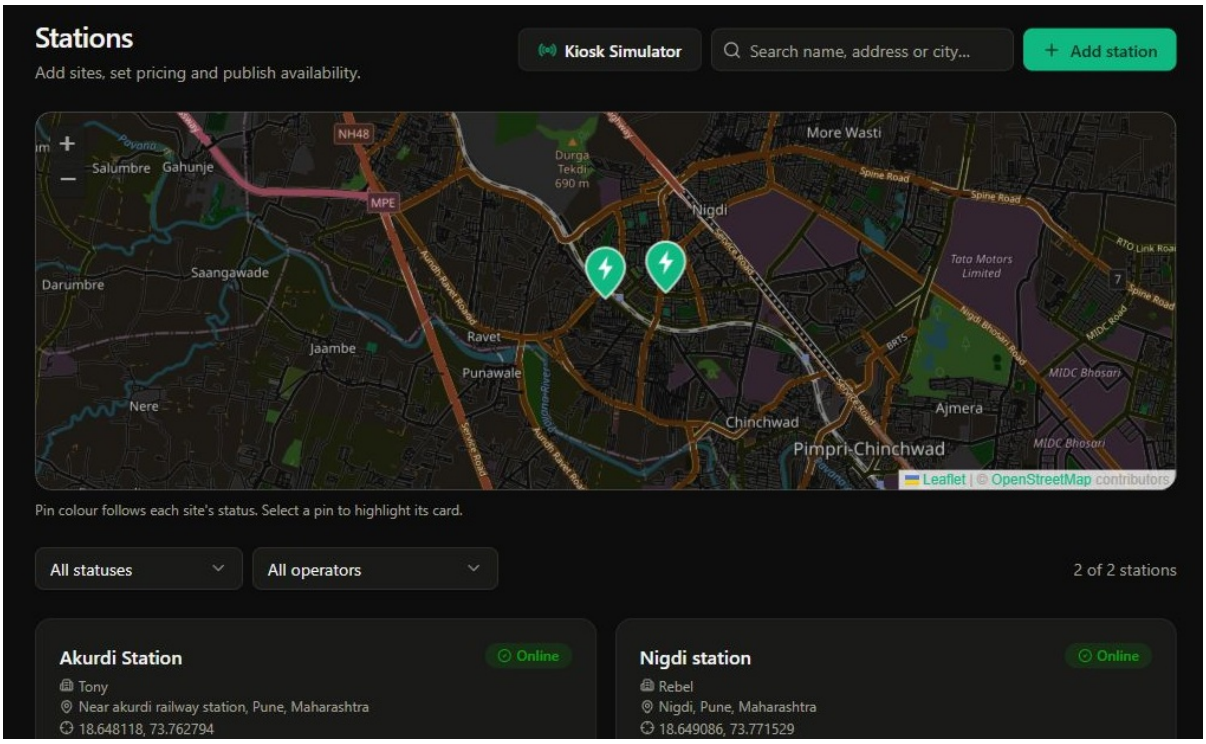


Figure 7.3 Station management and location view

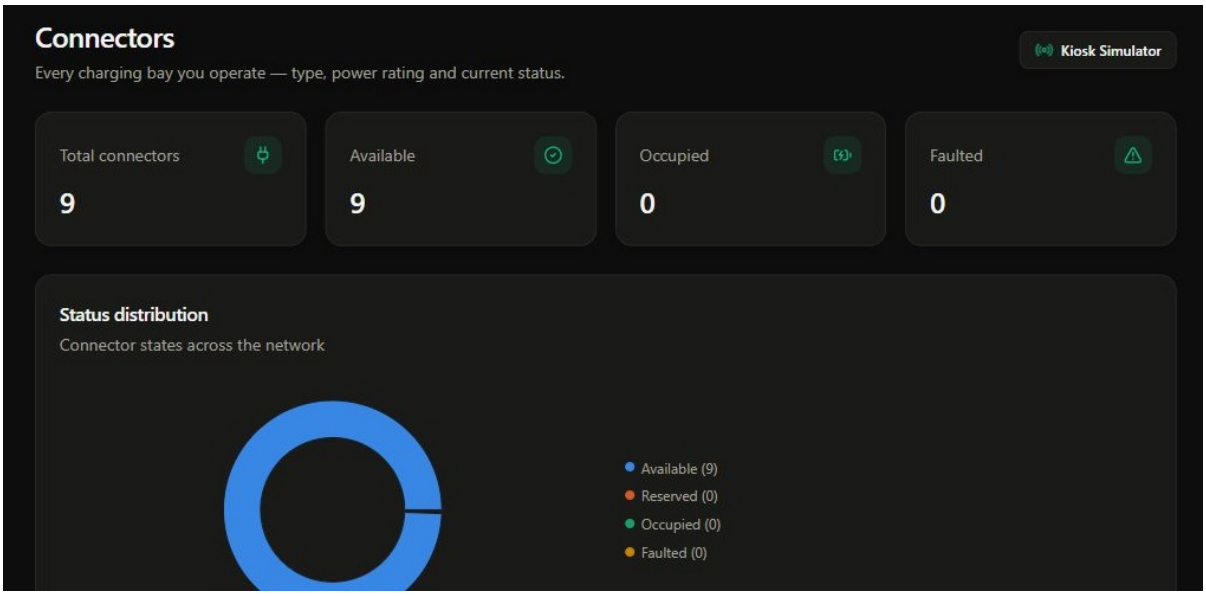


Figure 7.4 Connector status and availability board

CHAPTER 7: PROJECT FUNCTIONALITY SCREENSHOTS

7.3 Operator: Kiosk Terminal Simulation

Representative interface captures from project test data.

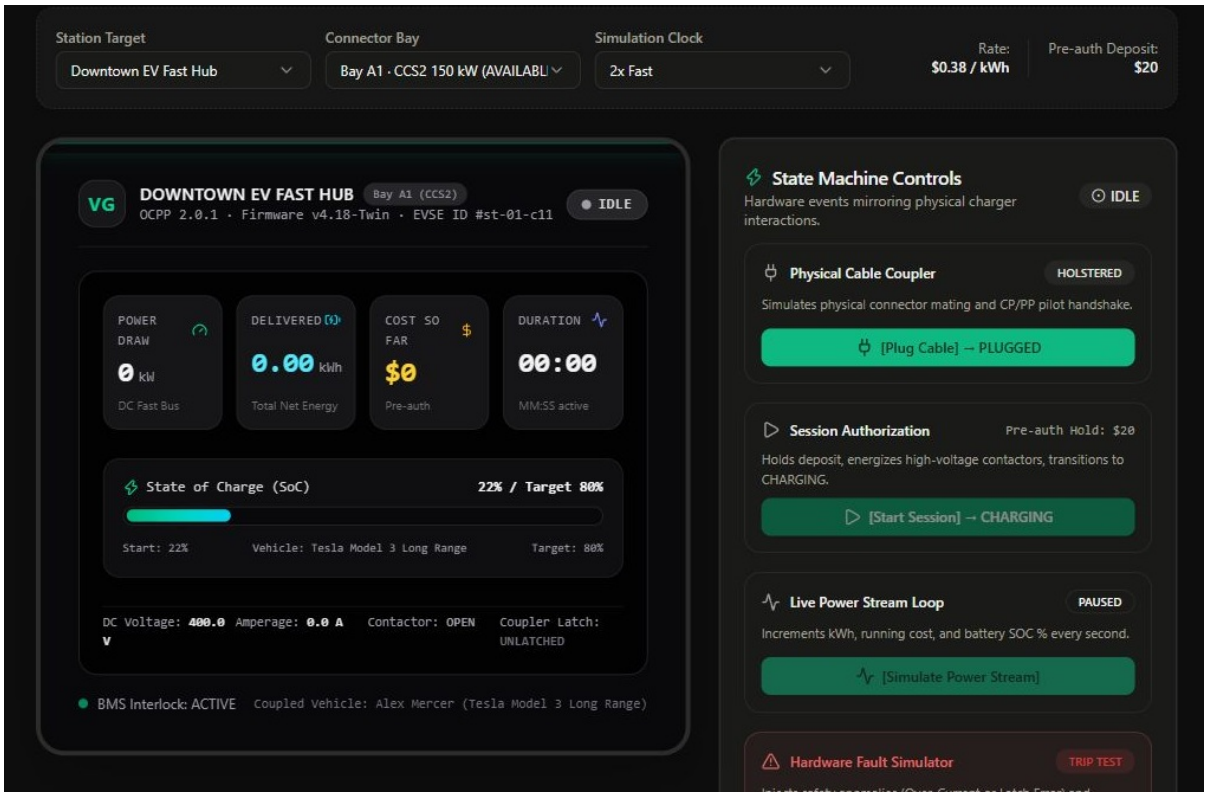


Figure 7.5 Kiosk telemetry and charging state controls

The kiosk is a software simulator; no physical charger is connected.

It models cable, session, power, and fault-state transitions.